Topic and contents

UNSW, School of Mathematics and Statistics

Profs Josef Dick and Kassem Mustapha

MATH3311/MATH5335

Topic 09 – Random numbers and simulation

Random numbers

Distinguish between

1. True random numbers, generated by a random physical process.
   - Example: radioactive decay (http://www.fourmilab.ch/hotbits/).
2. Pseudo-random numbers, generated by a deterministic algorithm.
   - Deterministic $\Longrightarrow$ reproducible.
   - Good statistical properties $\Longrightarrow$ statistically close to truly random.
   - Efficient $\Longrightarrow$ can generate large numbers of samples.
3. Quasi-random numbers, which are deterministic sequences designed to be well distributed in multidimensional space.

- General information: http://www.random.org/randomness/.
- MATLAB: `rand(m,n)`; Python `numpy.random.rand(m,n)`
  These return an $m \times n$ matrix whose entries are pseudo-random numbers uniformly distributed in $[0, 1]$.

Uniform random numbers

Let $X \sim U([a,b])$, that is, X is a random variable uniformly distributed on $[a,b]$. Then for any $a \leq \alpha \leq \beta \leq b$,

$$\text{Prob}\,(\alpha < X < \beta) = \frac{\beta - \alpha}{b-a} = \int_\alpha^\beta p(x)\,dx,$$

where the uniform probability density function (PDF) is

$$p(x) = \begin{cases} \dfrac{1}{b-a}, & a \leq x \leq b, \\ 0, & \text{otherwise.} \end{cases}$$

Theorem (Mean and variance of a uniform random variable)

If $X \sim U([a,b])$, then its mean and variance are

$$\mathbb{E}[X] = \int_a^b x p(x)\,dx = \frac{a+b}{2}, \quad \mathbb{V}ar[X] = \int_a^b (x - \mathbb{E}[X])^2 p(x)\,dx = \frac{(b-a)^2}{12}.$$

Linear change of variables

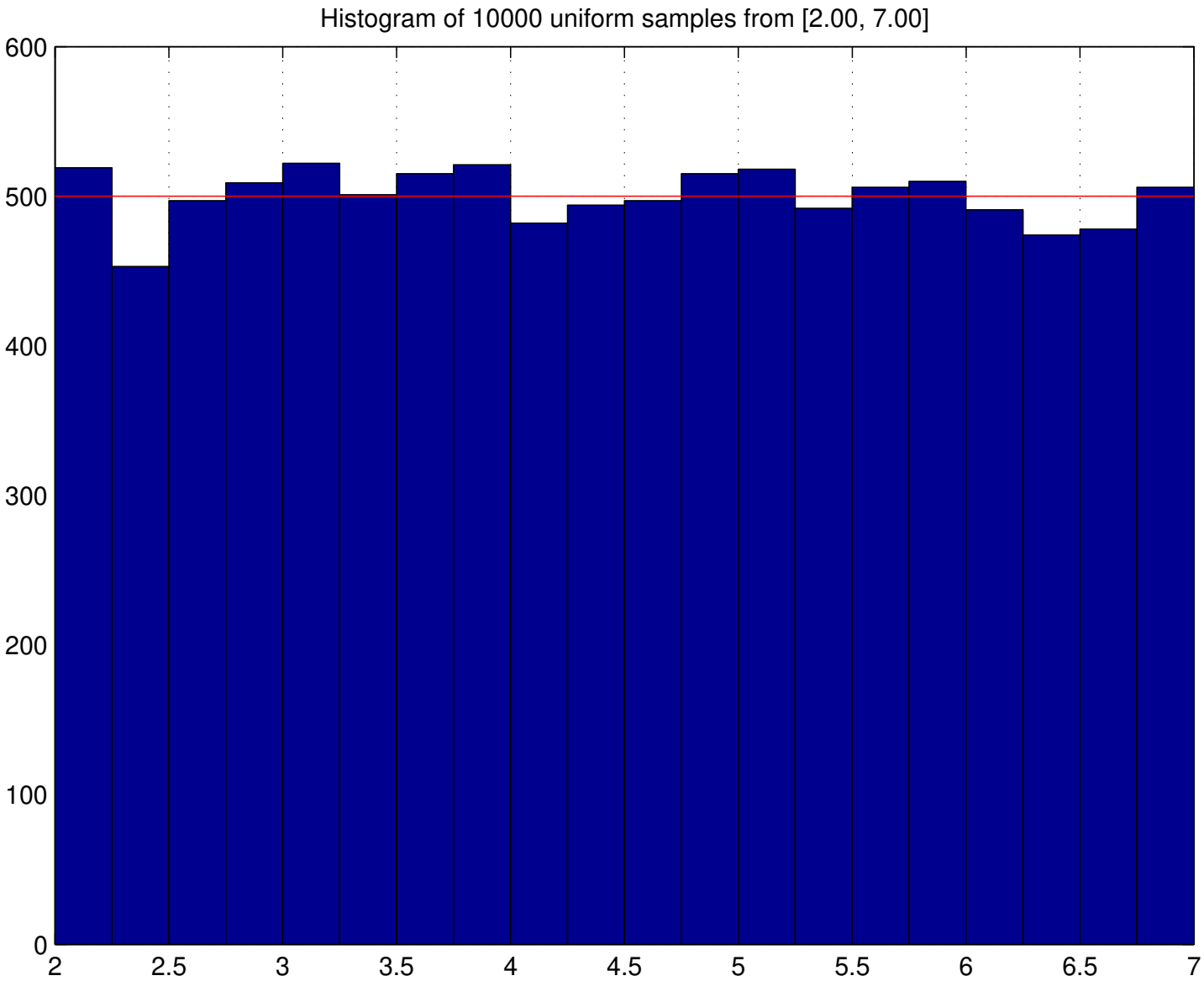
- If $X \sim U([0,1])$ then the linear change of variable

$$Y = (1 - X)a + Xb = a + (b - a)X$$

gives a random variable $Y \sim U([a,b])$.

Example (MATLAB `pseudorandom.m`; Python `pseudorandom.py`)

```
x = rand(10000,1);
a = 2; b = 7;
y = (1-x)*a + x*b;
mean(y) = 4.4978 !!    ((a+b)/2 = 4.5000)
std(y)  =   1.4397 !!  ((b-a)/sqrt(12)=1.4434)
histogram(y,20)
%  bins the elements of y into 20 equally spaced containers
% and returns the number of elements in each container
```


Histogram of 10000 uniform samples from [2.00, 7.00]
600
500
400
300
200
100
0
2
2.5
3
3.5
4
4.5
5
5.5
6
6.5
7

Inverse transform sampling

- Let Y be a random variable with PDF f and cumulative distribution function (CDF)

$$F(y) = \text{Prob}\,(Y \le y) = \int_{-\infty}^{y} f(u)\,du, \qquad -\infty < y < \infty.$$

- Assume that F is strictly increasing (for example, if $f(y) > 0$ for all y). Then, F^{-1} exists, and the transformed random variable $X = F(Y) \sim U([0,1])$ because for any $0 < \alpha < \beta < 1$,

$$\text{Prob}\,(\alpha < X < \beta) = \text{Prob}\,(\alpha < F(Y) < \beta) = \text{Prob}\,\left(F^{-1}(\alpha) < Y < F^{-1}(\beta)\right)$$

$$= \int_{F^{-1}(\alpha)}^{F^{-1}(\beta)} f(u)\,du = F(F^{-1}(\beta)) - F(F^{-1}(\alpha)) = \beta - \alpha,$$

- Conversely: given samples $x_1, x_2, \ldots, x_n \sim U([0,1])$, we generate samples from the distribution with CDF F by setting $y_i = F^{-1}(x_i)$, $i = 1, \ldots, n$. Indeed, $\text{Prob}\,(a < Y < b) =$

$$\text{Prob}\,\left(a < F^{-1}(X) < b\right) = \text{Prob}\,(F(a) < X < F(b)) = F(b) - F(a),$$

which shows that Y has CDF F.

Normal random variables

Let $Y \sim \mathcal{N}(\mu, \sigma^2)$, that is, Y is a random variable normally distributed with mean μ and variance σ^2.

- PDF:

$$p(y) = \frac{1}{\sqrt{2\pi}\,\sigma} \mathrm{e}^{-\frac{(y-\mu)^2}{2\sigma^2}}, \qquad -\infty < y < \infty.$$

  Hence,

$$\text{Prob}\,(a < Y < b) = \int_a^b p(y)\,dy.$$

- MATLAB: `randn(m,n)` and Python: `numpy.random.randn(m,n)` return an $m \times n$ matrix of pseudo-random numbers from the standard normal distribution $\mathcal{N}(0,1)$.

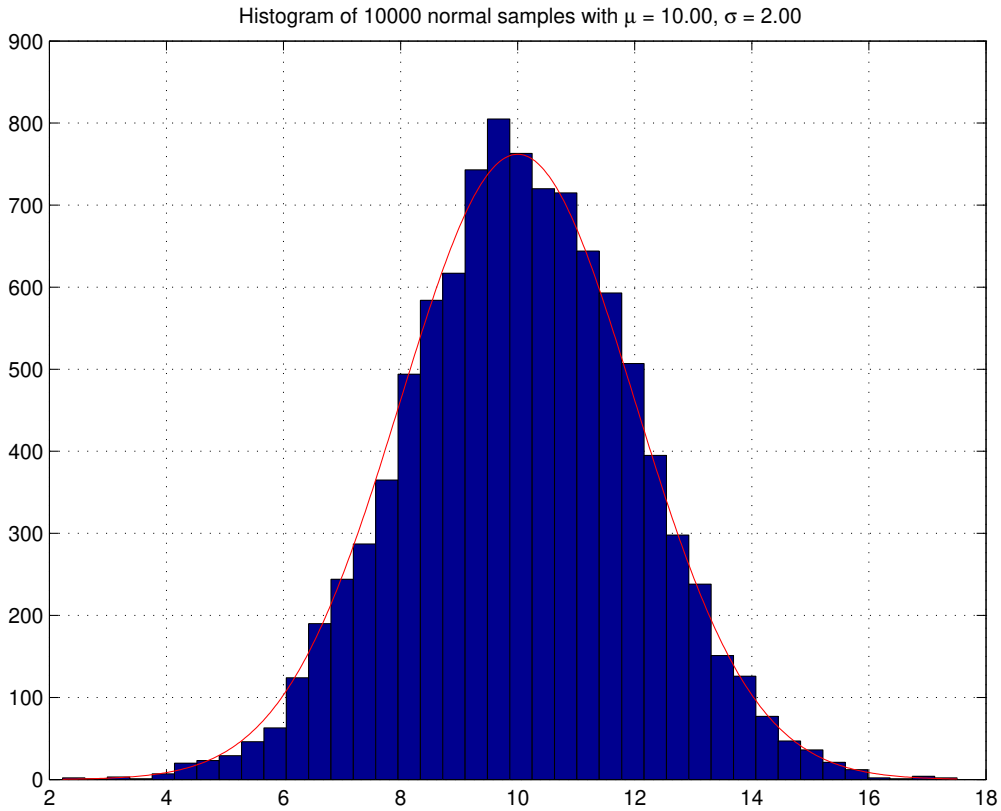
Note: If $X \sim \mathcal{N}(0,1)$ and $Y = \mu + \sigma X$, then $Y \sim \mathcal{N}(\mu, \sigma^2)$ because

$$\text{Prob}\,(a < Y < b) = \text{Prob}\left(\frac{a-\mu}{\sigma} < X < \frac{b-\mu}{\sigma}\right) = \frac{1}{\sqrt{2\pi}} \int_{\frac{a-\mu}{\sigma}}^{\frac{b-\mu}{\sigma}} \mathrm{e}^{-x^2/2}\,dx$$

$$= \frac{1}{\sqrt{2\pi}\,\sigma} \int_a^b \mathrm{e}^{-\frac{(y-\mu)^2}{2\sigma^2}}\,dy.$$

Example (MATLAB `pseudorandom.m`, Python `pseudorandom.py`)

```
mu = 10; sigma = 2; N = 10000; nbins = 40;
x = randn(N,1); y = mu + sigma * x;
yy = linspace(min(y), max(y), 200);
p = exp(-(yy-mu).^2/(2*sigma^2))/(sqrt(2*pi)*sigma);
histogram(y, nbins); plot(yy, c * p, 'r-')
```



Sums of independent random variables

Let $X_1, X_2, \ldots, X_n$ be independent and identically distributed (i.i.d.) random variables, each having the same distribution as X. Define the sum $Y = \sum_{i=1}^{n} X_i$. Then

$$\mathbb{E}[Y] = n\,\mathbb{E}[X], \qquad \mathbb{V}ar[Y] = n\,\mathbb{V}ar[X].$$

- If $X \sim \mathcal{N}(\mu, \sigma^2)$, then $Y \sim \mathcal{N}(n\mu, n\sigma^2)$.
- Even if X is not normally distributed, the Central Limit Theorem states that, for sufficiently large n, the distribution of Y is approximately normal. This idea can be used to generate approximately normal pseudo-random numbers.

Example: Let $X \sim U([-1, 1])$. Then $\mathbb{E}[Y] = 0$ and $\mathbb{V}ar[Y] = n\,\mathbb{V}ar[X] = \frac{n}{3}$. Thus, by the Central Limit Theorem, for sufficiently large n, the normalized sum

$$Z_n = \sqrt{\frac{3}{n}}\, Y$$

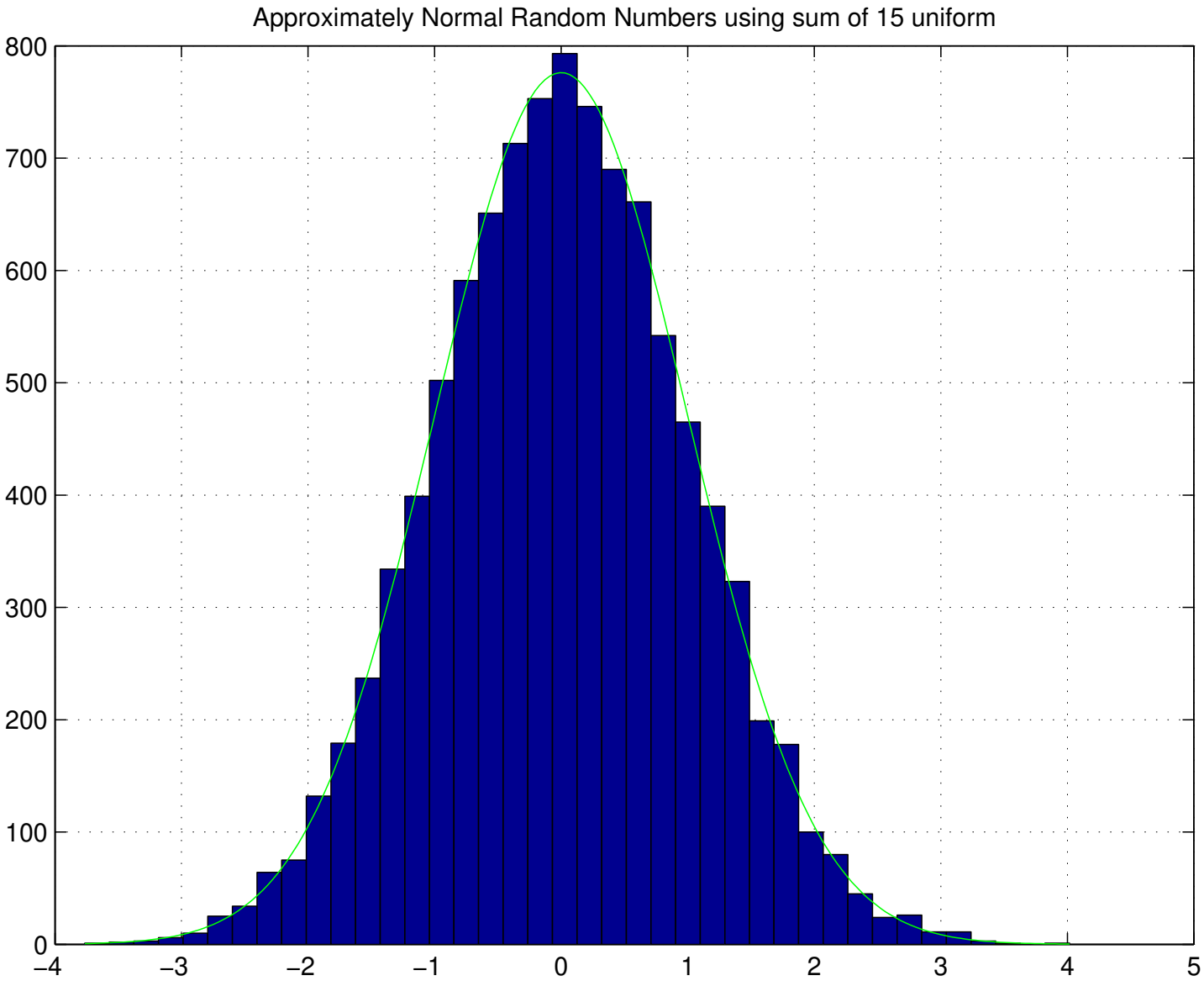
is approximately standard normally distributed.

Sums of independent random variables (continued)

Example (MATLAB `approxnormal.m`, Python `approxnormal.py`)

```
n = 15; N = 10000;
x = 2 * rand(n, N) - 1;
zn = sqrt(3/n) * sum(x);
mean(zn) = ??
std(zn) = ??
nbins = 40;
histogram(zn, nbins);
```

- Each row of `x` contains 10,000 pseudo-random numbers, uniformly distributed on $[-1, 1]$.
- `zn` contains 10,000 realizations (samples) of the normalized random variable Z_n.
- The histogram of `zn` is close to the standard normal density.


Approximately Normal Random Numbers using sum of 15 uniform
800
700
600
500
400
300
200
100
0
−4
−3
−2
−1
0
1
2
3
4
5

Monte-Carlo methods (MCMs)

- We aim to compute the high-dimensional expected value:

$$\mathbb{E}[f] := \mathbb{E}[f(\mathbf{X})] = \int_{\mathbb{R}^d} f(\mathbf{x})\, p(\mathbf{x})\, d\mathbf{x},$$

  where $p(\mathbf{x})$ is a PDF.
  - $\mathbf{X} = (X_1, X_2, \ldots, X_d)^T \in \mathbb{R}^d$ is a random vector,
  - $\mathbf{x} = (x_1, x_2, \ldots, x_d)^T \in \mathbb{R}^d$ is a point in the sample space,
  - $d\mathbf{x} = dx_1\, dx_2\, \ldots\, dx_d$,
  - For an event $\Omega \subseteq \mathbb{R}^d$,

$$\text{Prob}(\mathbf{X} \in \Omega) = \int_{\Omega} p(\mathbf{x})\, d\mathbf{x}.$$

- The MCM approximates $\mathbb{E}[f(\mathbf{X})]$ by the sample mean $\mathbb{E}_N[f] = \frac{1}{N}\sum_{i=1}^{N} f(\mathbf{x}_i)$.
  - $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N$ are independent samples drawn from the distribution with PDF $p(\mathbf{x})$.
  - $\mathbb{E}(\mathbb{E}_N[f]) = \frac{1}{N}\sum_{i=1}^{N} \mathbb{E}(f(\mathbf{x}_i)) = \int_{\mathbb{R}^d} f(\mathbf{x})p(\mathbf{x})\, d\mathbf{x} = \mathbb{E}[f]$.

  Therefore, $\mathbb{E}_N[f]$ is an unbiased estimator of $\mathbb{E}[f]$.

Monte-Carlo methods (continued)

- The Monte-Carlo error is the random variable $\epsilon_N[f] = \mathbb{E}[f] - \mathbb{E}_N[f]$.
- The root mean square (RMS) error is $e_N[f] = \sqrt{\mathbb{E}[(\epsilon_N[f])^2]}$.

Since $\mathbb{E}_N[f]$ is an unbiased estimator, $\mathbb{E}[\mathbb{E}_N[f]] = \mathbb{E}[f]$, and hence

$$e_N^2[f] = \mathbb{E}\left[\left(\mathbb{E}[f] - \frac{1}{N}\sum_{i=1}^{N} f(\mathbf{x}_i)\right)^2\right]$$

$$= (\mathbb{E}[f])^2 - 2\frac{\mathbb{E}[f]}{N}\sum_{i=1}^{N}\mathbb{E}[f(\mathbf{x}_i)] + \frac{1}{N^2}\sum_{i,j=1}^{N}\mathbb{E}[f(\mathbf{x}_i)f(\mathbf{x}_j)]$$

$$= -(\mathbb{E}[f])^2 + \frac{1}{N^2}\sum_{i=1}^{N}\mathbb{E}\left[f^2(\mathbf{x}_i)\right] + \frac{1}{N^2}\sum_{i,j=1_{i\neq j}}^{N}\mathbb{E}[f(\mathbf{x}_i)]\,\mathbb{E}[f(\mathbf{x}_j)]$$

$$= -(\mathbb{E}[f])^2 + \frac{1}{N}\mathbb{E}\left[f^2\right] + \frac{N^2-N}{N^2}(\mathbb{E}[f])^2 = \frac{1}{N}(\mathbb{E}\left[f^2\right] - (\mathbb{E}[f])^2) = \frac{1}{N}\mathbb{V}ar[f].$$

Therefore, $e_N[f] = \sigma[f]/N^{1/2} = O(N^{-1/2})$, is independent of the dimension d.

Monte-Carlo methods (continued)

- Unlike tensor-product quadrature rules, the convergence rate of Monte Carlo methods does not deteriorate as the dimension d increases.
- Since the Monte-Carlo estimator is a random variable, its error is also random. Consequently, although the RMS error is $O(N^{-1/2})$, the error in any particular Monte Carlo simulation may be significantly larger (or smaller) than its RMS value.

Example

Compute a Monte-Carlo approximation with $N = 10^6$ to the 2D integral

$$\int_0^1 \int_0^1 \frac{dx\, dy}{1+x+y} = 3\log 3 - 4\log 2 = 0.523248\ldots.$$

MATLAB `mcex1.m`, Python `mcex1.py`
Running these codes ten times (using different pseudo-random numbers) produced the results shown on the next slide.

Monte-Carlo methods (continued)

Trial	MCM solution	Error
1	0.52344	1.92e-04
2	0.52302	-2.28e-04
3	0.52338	1.27e-04
4	0.52350	2.48e-04
5	0.52337	1.19e-04
6	0.52328	2.93e-05
7	0.52316	-8.71e-05
8	0.52336	1.12e-04
9	0.52324	-3.90e-06
10	0.52342	1.73e-04

The average Monte Carlo estimate is $0.52331629\ldots$, which is close to the exact value $0.523248\ldots$.

- If N is increased by a factor of 4, then the RMS error is reduced, on average, by a factor of 2. However, for any particular Monte-Carlo simulation, the error may increase because the estimator is random.

Monte-Carlo methods (continued)

Example (Revisiting the $d = 50$ example)

Assume that $\sigma[f] = 10$ and that one clock cycle is required to evaluate f at each sample point. How long would it take to approximate $\mathbb{E}[f]$ for $d = 50$ using the MCM, if we run the program on a 3 GHz quad-core PC and require an RMS error less than 0.01?

- Speed of the computer: $4 \times 3 \times 10^9 = 1.2 \times 10^{10}$ clock cycles per second.
- Since
$$\frac{\sigma[f]}{\sqrt{N}} = \frac{10}{\sqrt{N}} \leq 10^{-2},$$
we require $\sqrt{N} \geq 10^3$. If $N = 10^6$, then the total CPU time to evaluate f at the 10^6 sample points is
$$10^6/(1.2 \times 10^{10}) \approx 8.3 \times 10^{-5} \text{ seconds.}$$
- In comparison, the tensor-product Simpson rule required approximately
$$2.35 \times 10^{17} \text{ years.}$$

Variance reduction (control variates)

- We wish to estimate $\mathbb{E}[f]$, but the variance $\sigma(f)$ is large.
- Suppose we know a function g such that
  - $g \approx f$,
  - $\mathbb{E}[g] = \mu_g$ is known analytically,
  - $\sigma(f-g) \ll \sigma(f)$.
- Using the linearity of expectation,

$$\mathbb{E}[f] = \mathbb{E}[(f-g)+g] = \mathbb{E}[f-g] + \mathbb{E}[g] = \mathbb{E}[f-g] + \mu_g.$$

- Hence, we estimate $\mathbb{E}[f-g]$ by MCM and then add the known quantity μ_g.
- The Monte-Carlo methods now depends on $\sigma(f-g)$, which is much smaller than $\sigma(f)$.
- The convergence rate remains $O(N^{-1/2})$, but the constant is much smaller because $\sigma(f-g) \ll \sigma(f)$.

Antithetic variables

- Suppose the integrand (including any PDF) is an odd function, that is,

$$f(-\mathbf{x}) = -f(\mathbf{x}) \qquad \text{for all } \mathbf{x} \in \mathbb{R}^d.$$

  Then

$$\int_{\mathbb{R}^d} f(\mathbf{x})\, d\mathbf{x} = 0.$$

- The antithetic variables technique uses pairs of sample points. Assume that N is even and choose

$$\mathbf{x}_{N/2+j} = -\mathbf{x}_j, \qquad j = 1, \ldots, \frac{N}{2}.$$

  Then, whenever f is odd (symmetric about the origin), $\frac{1}{N}\sum_{i=1}^{N} f(\mathbf{x}_i) = 0$.
- For a general function f, write $f = g + h$, where g is odd. Then, $\int_{\mathbb{R}^d} f(\mathbf{x})\, d\mathbf{x} = \int_{\mathbb{R}^d} h(\mathbf{x})\, d\mathbf{x}$ and thus, antithetic variables integrate the odd component exactly. Note: We do not need to know g, just generate the sample points as above.
- On the cube $[0,1]^d$, symmetry is taken about the center $\mathbf{x}_c = \frac{1}{2}(1, 1, \ldots, 1) = \frac{1}{2}\mathbf{e}$. Hence we choose $\mathbf{x}_{N/2+j} = \mathbf{e} - \mathbf{x}_j$, $j = 1, \ldots, \frac{N}{2}$.

Another variance reduction method (importance sampling)

- We have

$$\mathbb{E}[f] = \mathbb{E}[f(\mathbf{X})] = \int_{\mathbb{R}^d} f(\mathbf{x})p(\mathbf{x})\,d\mathbf{x} = \int_{\mathbb{R}^d} \frac{f(\mathbf{x})}{g(\mathbf{x})} g(\mathbf{x})p(\mathbf{x})\,d\mathbf{x},$$

  where

$$g \approx f \quad \text{with } g(\mathbf{x}) \neq 0 \text{ whenever } f(\mathbf{x}) \neq 0,$$

  and

$$q(\mathbf{x}) = g(\mathbf{x})p(\mathbf{x})$$

  is a PDF.
- Generate independent samples $\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_N$ from the probability density $q(\mathbf{x}) = g(\mathbf{x})p(\mathbf{x})$.
- Estimate

$$\mathbb{E}[f] \approx \frac{1}{N} \sum_{i=1}^{N} \frac{f(\mathbf{y}_i)}{g(\mathbf{y}_i)}.$$

- Variance of $\frac{f}{g}$ is expected to be much small than the variance of f.

Quasi-Monte Carlo methods (QMCMs)

Approximate a d-dimensional integral by the equally weighted sum

$$\int_{[0,1]^d} f(\mathbf{x})\, d\mathbf{x} \approx \frac{1}{N} \sum_{j=1}^{N} f(\mathbf{x}_j),$$

where the points $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N$ are deterministic (not random).

Koksma–Hlawka inequality

$$\left| \int_{[0,1]^d} f(\mathbf{x})\, d\mathbf{x} - \frac{1}{N} \sum_{j=1}^{N} f(\mathbf{x}_j) \right| \leq V(f)\, D_N^*(\mathbf{x}_1, \ldots, \mathbf{x}_N).$$

- $V(f)$ measures the variation of the integrand f and $D_N^*(\mathbf{x}_1, \ldots, \mathbf{x}_N)$ is the star discrepancy of the point set:

$$D_N^*(\mathbf{x}_1, \ldots, \mathbf{x}_N) = \sup_{\mathbf{y} \in [0,1]^d} \left| \prod_{i=1}^{d} y_i - \frac{\#\{\mathbf{x}_j \in [\mathbf{0}, \mathbf{y})\}}{N} \right|.$$

  - The volume of the rectangle $[\mathbf{0}, \mathbf{y})$ is $\prod_{i=1}^{d} y_i$.
  - Discrepancy between volume of (d-dimensional) rectangle with corners $\mathbf{0}$ and $\mathbf{y}$ and fraction of points lying in this rectangle.

Proof in 1-dimension. Assume that f is absolutely continuous on $[0,1]$. Then,

$$f(x) = f(1) - \int_x^1 f'(y)dy. \text{ Thus,}$$

$$\int_0^1 f(x)dx - \frac{1}{N}\sum_{j=1}^N f(x_j) = f(1) - \int_0^1 \int_x^1 f'(y)dydx - \frac{1}{N}\sum_{j=1}^N \left(f(1) - \int_{x_j}^1 f'(y)dy\right)$$

$$= -\int_0^1 \int_0^y f'(y)dxdy + \frac{1}{N}\sum_{j=1}^N \int_0^1 f'(y)1_{[x_j,1]}(y)dy$$

$$= -\int_0^1 f'(y)\left(y - \frac{1}{N}\sum_{j=1}^N 1_{[x_j,1]}(y)\right)dy. \text{ Therefore,}$$

$$\left|\int_0^1 f(x)dx - \frac{1}{N}\sum_{j=1}^N f(x_j)\right| \le \underbrace{\int_0^1 |f'(y)|\,dy}_{V(f)} \underbrace{\sup_{y\in[0,1]}\left|y - \frac{\#x_j \in [0,y]}{N}\right|}_{D_N^*(\mathbf{x}_1,\ldots,\mathbf{x}_N)}$$

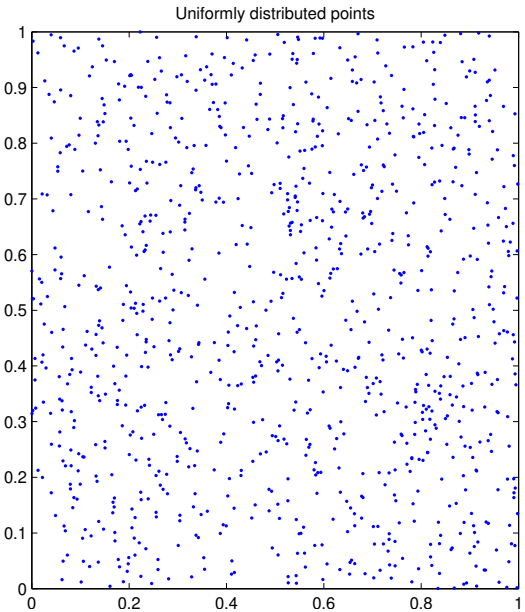
- D_N^* measures the discrepancy between the length of the interval $[0,y)$ and the fraction of sample points lying in $[0,y)$.

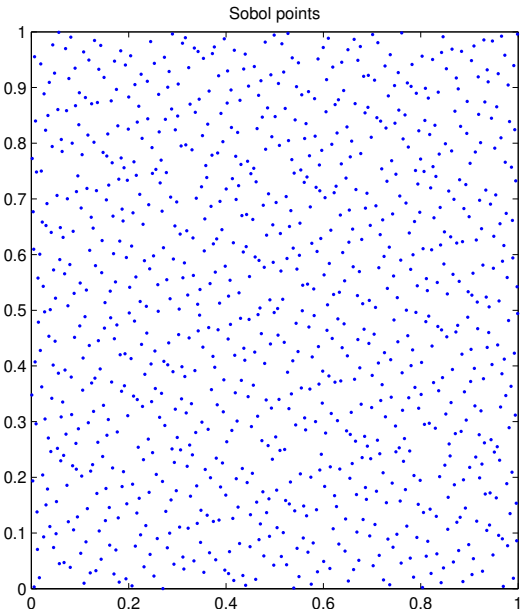
Low-discrepancy sequences

- In QMCMs, we choose deterministic points $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N$ so that the star discrepancy is as small as possible.
- Well-distributed (low-discrepancy) point sets satisfy

$$D_N^*(\mathbf{x}_1, \ldots, \mathbf{x}_N) \leq C \frac{(\log N)^d}{N},$$

  where the constant C depends on the dimension d, but not on N.
- Common low-discrepancy sequences include Faure, Halton, Hammersley, Niederreiter, and Sobol.





Quasi-Monte Carlo using Sobol points

Example (QMC using Sobol points)

The QMC approximation with $N = 100 \times 2^j$, $j = 0, \ldots, 20$, uses the first N Sobol points to approximate the two-dimensional integral

$$\int_0^1 \int_0^1 \frac{dx\, dy}{1+x+y} = 3\log 3 - 4\log 2 = 0.523248143764548\ldots.$$

MATLAB `qmcex1.m`, Python `qmcex1.py`